Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет информационных технологий и программирования
Кафедра прикладной информатики

ЛАБОРАТОРНАЯ РАБОТА №1

Реализация REST API на основе boilerplate

Выполнил: студент группы К3341
Кижваткин Н.М.

Санкт-Петербург
2025

1. ЦЕЛЬ И ЗАДАЧИ

Цель работы: реализация REST API информационной системы Callmentor на основе разработанного boilerplate (шаблона FastAPI-приложения).

Задачи:

1. Изучить структуру шаблона FastAPI-приложения.
2. Реализовать модели данных и схемы Pydantic.
3. Реализовать CRUD-операции для основных сущностей системы.
4. Настроить аутентификацию и авторизацию.
5. Обеспечить автоматическую генерацию OpenAPI-документации.

2. ОПИСАНИЕ BOILERPLATE

В репозитории проекта присутствует директория templates/fastapi_template, содержащая шаблон FastAPI-приложения. Структура шаблона отражает лучшие практики организации кода FastAPI-проекта:

Таблица 1 — Структура шаблона FastAPI

Директория/файл	Назначение
app/api/v1/	Роутеры API версии 1
app/core/	Бизнес-логика (use cases)
app/models/	SQLAlchemy ORM-модели
app/repository/	Слой работы с БД
app/schemas/	Pydantic-схемы (DTO)
app/services/	Сервисный слой
app/dependencies/	Зависимости FastAPI (DI)
app/utils/	Вспомогательные утилиты
app/config.py	Конфигурация через pydantic-settings
app/main.py	Точка входа приложения

3. РЕАЛИЗОВАННАЯ АРХИТЕКТУРА ПРИЛОЖЕНИЯ

3.1. Многоуровневая архитектура

В проекте Callmentor реализована многоуровневая архитектура, соответствующая шаблону:

– Уровень маршрутизации (api/apps/*): обработка HTTP-запросов, валидация входных параметров, вызов бизнес-логики, формирование HTTP-ответов.

– Уровень бизнес-логики (core/*): реализация use cases, оркестрация операций с БД и внешними сервисами.
– Уровень данных (models/*): ORM-модели SQLAlchemy, миграции Alembic.
– Уровень шаблонов (templates/*): Pydantic-модели запросов/ответов, перечисления.
– Уровень утилит (utils/*): вспомогательные классы для работы с БД, аутентификацией, внешними API.

3.2. Точка входа (gateway/main.py)

Приложение инициализируется через lifespan-контекстный менеджер, который при запуске выполняет: применение миграций Alembic (alembic upgrade head), инициализацию данных (must_init), запуск фоновых задач BitrixDownloaderScheduler, запуск планировщика APScheduler с регулярными задачами. Middleware регистрируются в обратном порядке: сначала CORSMiddleware (разрешает все источники), затем PermissionMiddleware (проверяет права доступа).

3.3. Реализация CRUD-операций

Для каждого ресурса реализованы отдельные файлы эндпоинтов по HTTP-методу: get_*.py, post_*.py, put_*.py, patch_*.py, delete_*.py. Это обеспечивает чёткое разделение ответственности и упрощает навигацию по коду. Каждый файл содержит роутер и функции-обработчики, делегирующие логику в соответствующие модули core/.

3.4. Работа с базой данных

Для взаимодействия с PostgreSQL используется SQLAlchemy 2.0 в асинхронном режиме (asyncpg). Класс DatabaseWorkerAsync (utils/internal_workers/database_worker.py) предоставляет универсальный интерфейс для ORM-операций: custom_orm_select, count, insert, update, delete. Все операции выполняются в асинхронных транзакциях через контекстный менеджер async with session.

3.5. Конфигурация приложения

Конфигурация вынесена в классы на основе pydantic-settings: ConfigApp (общие настройки: SECRET_KEY, ALGORITHM, REDIS_DSN), ConfigDataBase (PostgreSQL DSN), ConfigCall (URL ML-сервиса), ConfigEmail, ConfigSuperAdmin. Все значения загружаются из .env-файлов, что соответствует принципу Twelve-Factor App.

4. КЛЮЧЕВЫЕ КОМПОНЕНТЫ РЕАЛИЗАЦИИ

4.1. Аутентификация (AuthNamespace)

Класс AuthNamespace (utils/internal_workers/auth_worker.py) реализует полный цикл аутентификации: хэширование паролей через bcrypt, генерацию и верификацию JWT-токенов через python-jose, проверку прав доступа к маршрутам. Метод verify_route_access извлекает текущий URL из объекта Request и проверяет наличие соответствующей записи в таблице m2m_role_routes для роли пользователя.

4.2. Планировщик задач (APScheduler)

Фоновые задачи выполняются через AsyncIOScheduler из библиотеки APScheduler:

- daily_minutes_log_task — ежедневное в 00:05 UTC сохранение использования тарифных минут (запись в seconds_usage_log).
- weekly_kpi_report_task — каждое воскресенье в 02:00 UTC генерация KPI-отчёта по менеджерам через LLM.
- weekly_objections_clustering_task — каждое воскресенье в 02:30 UTC кластеризация возражений клиентов.
- backtrack_bitrix_calls_task — каждые 3 дня проверка и дозагрузка пропущенных звонков из Bitrix24.

4.3. Модуль работы с звонками (CallNamespace)

Класс CallNamespace (utils/external_workers/call/call_worker.py) реализует взаимодействие с ML-сервисом: handle_process_task — загрузка файла и создание задачи на анализ, handle_check_task — опрос статуса выполнения, handle_send_waiting_task — повторная отправка звонка на обработку. Взаимодействие с ML-сервисом происходит по HTTP через httpx.

5. ТЕСТИРОВАНИЕ API

API доступен по двум адресам документации: /docs — Scalar UI с поддержкой Bearer-аутентификации (внутренний API) и /docs-user — Scalar UI с API-ключом (публичный API v1). Для тестирования использовался инструмент Postman. Среди протестированных сценариев: регистрация компании и пользователя, вход и получение токена, загрузка тестового аудиофайла, опрос статуса обработки, просмотр результатов анализа в дашборде.

6. ВЫВОДЫ

В ходе выполнения лабораторной работы реализован полноценный REST API системы Callmentor. На основе шаблона FastAPI создано многоуровневое приложение с чёткой архитектурой, асинхронной работой с PostgreSQL, JWT-аутентификацией и ролевой моделью доступа. Реализованы эндпоинты для всех основных функций системы: управления звонками, промтами, дашбордов и отчётов. Автоматическая документация через Scalar UI упрощает тестирование и интеграцию.